Programarea calculatoarelor și limbaje de programare – 4

-Automat de Băuturi-

~VHDL~

- **Emin Esra**
- **Ghiță Mihai-Andrei**
- **Ghinea Ioana-Mara**
- **Popescu Maria-Teodora**

GRUPA 322CC

DESCRIERE PROIECT

Am proiectat un automat de băuturi pe 8 biți, care oferă clientului două opțiuni de băuturi și acceptă bancnote de 1 leu, 5 lei și 10 lei. Pentru implementarea automatului am folosit limbajul de programare VHDL.

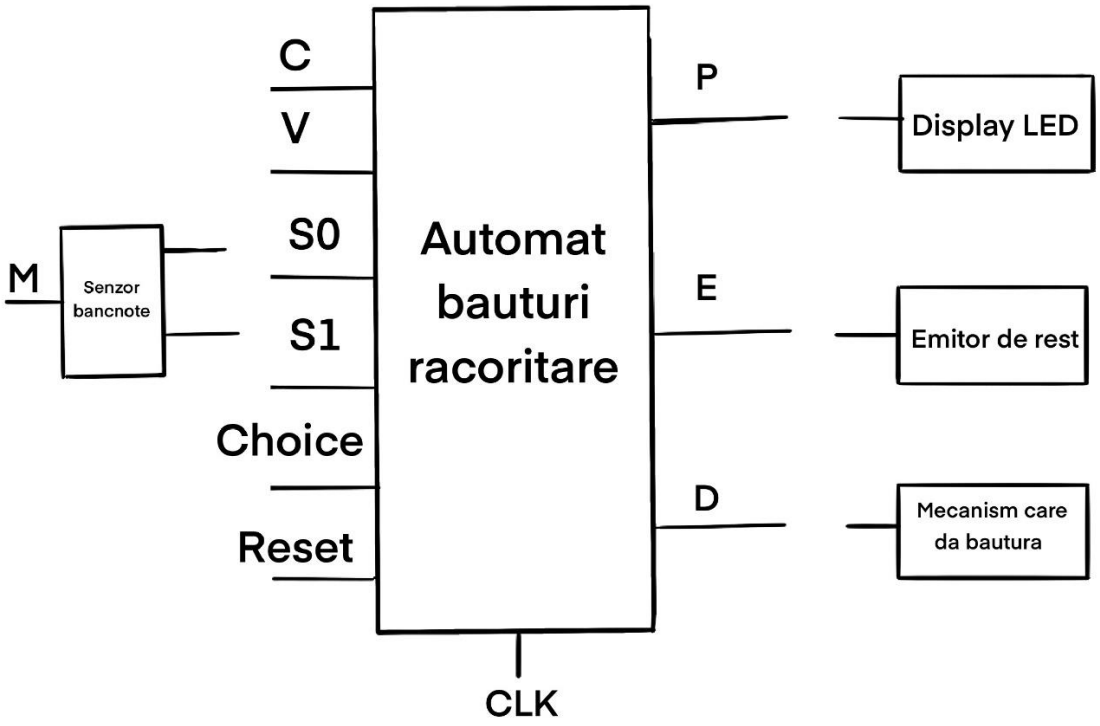
Iată cum ar trebui să funcționeze automatul:

Aparatul are un singur slot pentru bancnote care acceptă bancnote de un leu, 5 lei și 10 lei pe rând. Un senzor de bancnote oferă procesorului o intrare de 1 bit C care devine 1 atunci când este detectată o bancnote și o intrare de 8 biți V indicând valoarea bancnotei în lei. Două intrări de 8 biți SO și S1 indică costul celor două tipuri de băuturi. Se poate observa că valoarea SO și S1 poate fi setată de proprietarul automatului. Alegerea este o intrare de 1 bit către automatul care indică tipul de băutură selectat (Alegerea 0 este pentru băutura de tip 0 cu costul S0, iar Alegerea 1 este pentru băutura de tip 1 cu costul de S1).

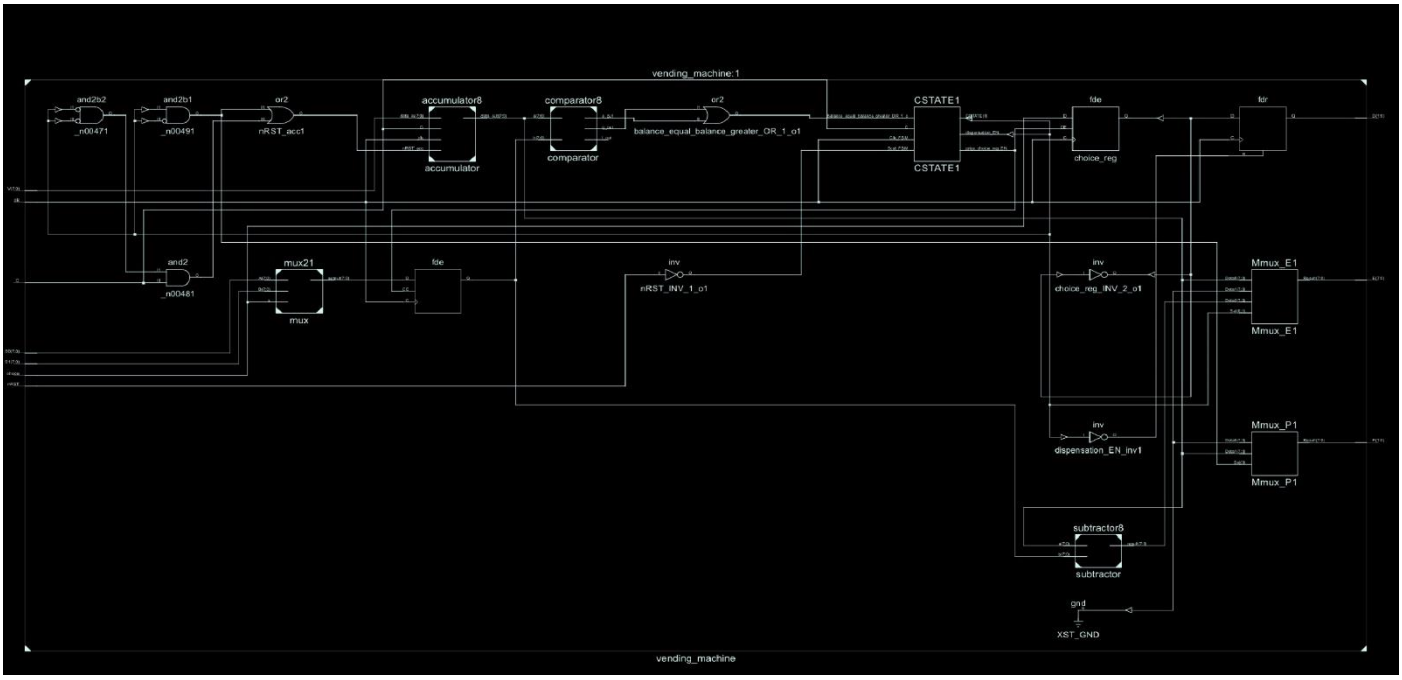
Dacă suma de bani depusă este mai mică decât costul băuturii, procesorul generează o ieșire de 8 biți P pentru a afișa suma de bani depusă. Odată ce procesorul a detectat bancnote a căror valoare este egală cu sau depășește costul băuturii, procesorul ar trebui să seteze o ieșire de 2 biți D pentru un ciclu de ceas, care determină distribuirea unei băuturi (00 sau 11 pentru nicio acțiune, 01 pentru eliberare de băutură răcoritoare tip 0 și 10 pentru eliberare de băutură răcoritoare tip 1).

De asemenea, automatul ar trebui să genereze ieșirea de 8 biți E care indică ca restul să fie returnat în bancnote.

SCHEMA BLOC



CONECTAREA CIRCUITELOR LOGICE



COD SURSĂ

```
--automat de bauturi
library ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity vending_machine is
    port(
        nRST : in std_logic;  --resetare negativa
        clk : in std_logic;
        C : in std_logic; --senzor bancnote: devine 1 cand detecteaza o bancnota
        V : in std_logic_vector(7 downto 0); --primeste valorea bancnotei de la senzor
        S0 : in std_logic_vector(7 downto 0); --pretul pentru prima bautura(acesta poate
fi definit de proprietar)
        S1 : in std_logic_vector(7 downto 0); --pretul pentru a doua bautura(acesta poate
fi definit de proprietar)
        choice : in std_logic; --alegere
        P : out std_logic_vector(7 downto 0); --ecran
        E : out std_logic_vector(7 downto 0); --rest
        D : out std_logic_vector(1 downto 0) --dispensare bautura
        );
end vending_machine;

architecture rtl of vending_machine is
component accumulator8 is
    port(
        clk: in std_logic;
        nRST_acc: in std_logic;
        C : in std_logic;   --devine 1 cand detecteaza o bancnota
        data_in : in std_logic_vector(7 downto 0);
```

```
        data_out : out std_logic_vector(7 downto 0)
        );
end component;

component subtractor8 is
    port(
        a : in std_logic_vector(7 downto 0);
        b : in std_logic_vector(7 downto 0);
        result : out std_logic_vector(7 downto 0)
        );
end component;

component comparator8 is
    port(
        a : in std_logic_vector(7 downto 0);
        b : in std_logic_vector(7 downto 0);
        g_out : out std_logic;
        e_out : out std_logic;
        l_out : out std_logic);
end component;

component mux21 is
    port(
        A : in std_logic_vector(7 downto 0);
        B : in std_logic_vector(7 downto 0);
        s : in std_logic;
        output : out std_logic_vector(7 downto 0)
        );
end component;

type FSMTYPE is (INIT_STATE, Coin_Reception, soda_dispensation);

signal CSTATE, NSTATE : FSMTYPE;
signal balance, price, price_reg, coins_to_return : std_logic_vector(7 downto 0);
signal price_choice_reg_EN, balance_greater, balance_equal, balance_lower: std_logic;
signal choice_reg, dispensation_EN : std_logic;
signal nRST_acc : std_logic;

begin
    price_registration : process( CLK )
    begin
        if (CLK'event and CLK = '1') then
            if (price_choice_reg_EN = '1') then
                price_reg <= price;
                choice_reg <= choice;
            end if ;
        end if ;
    end process ; --inregistrare pret


    state_registration : process( CLK )
    begin
        if (CLK'event and CLK = '1') then
            if (nRST = '0') then
```

```
                CSTATE <= INIT_STATE;
            else
                CSTATE <= NSTATE;
            end if ;
        end if ;
    end process ; --status

    soda_dispensation_proc: process(clk)
    begin
        if (CLK'event and CLK = '1') then
            if (dispensation_EN = '1') then
                if(choice_reg = '0') then
                    D <= "01";  --S0
                else
                    D <= "10";  --S1
                end if;
            else
                D <= "00";
            end if;
        end if;
    end process; --procesul de dispensare a bauturii

    next_state : process( CSTATE, balance, C, balance_equal, balance_greater,
coins_to_return)
    begin
        NSTATE <= CSTATE;
        nRST_acc <= '1';
        price_choice_reg_EN <= '0';
        dispensation_EN <= '0';
        p <= (others => '0');
        E <= (others => '0');

        case( CSTATE ) is
            when INIT_STATE =>
                nRST_acc <= '0';
                price_choice_reg_EN <= '1';
                E <= balance;
                if (C = '1') then
                    nRST_acc <= '1';
                    NSTATE <= Coin_Reception;
                end if ;

            when Coin_Reception =>
                P <= balance;
                if (balance_equal = '1' or balance_greater = '1') then
                    NSTATE <= soda_dispensation ;
                end if ;

            when soda_dispensation =>
                dispensation_EN <= '1';
                E <= coins_to_return;
                nRST_acc <= '0';
                NSTATE <= INIT_STATE;
```

```
            when others =>
        end case ;
    end process ; --urmatoarea stare

    mux : mux21 port map(S0, S1, choice, price);
    accumulator : accumulator8 port map (clk, nRST_acc, C, V, balance);
    comparator : comparator8 port map (balance, price_reg, balance_greater, balance_equal,
balance_lower);
    subtractor : subtractor8 port map (balance, price_reg, coins_to_return);

end rtl;
```

COD TESTBENCH

```
library ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

ENTITY vending_machine_TB IS
END vending_machine_TB;

ARCHITECTURE behavior OF vending_machine_TB IS

    -- Declararea componentelor pentru unitatea de test

    COMPONENT vending_machine
    PORT(
         nRST : IN  std_logic;
         clk : IN  std_logic;
         C : IN  std_logic;
         V : IN  std_logic_vector(7 downto 0);
         S0 : IN  std_logic_vector(7 downto 0);
         S1 : IN  std_logic_vector(7 downto 0);
         choice : IN  std_logic;
         P : OUT  std_logic_vector(7 downto 0);
         E : OUT  std_logic_vector(7 downto 0);
         D : OUT  std_logic_vector(1 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal nRST : std_logic := '0';
   signal clk : std_logic := '0';
   signal C : std_logic := '0';
   signal V : std_logic_vector(7 downto 0) := (others => '0');
   signal S0 : std_logic_vector(7 downto 0) := (others => '0');
   signal S1 : std_logic_vector(7 downto 0) := (others => '0');
   signal choice : std_logic := '0';
```

```
  --Outputs
   signal P : std_logic_vector(7 downto 0);
   signal E : std_logic_vector(7 downto 0);
   signal D : std_logic_vector(1 downto 0);

   -- Definirea perioadei de ceas
   constant clk_period : time := 10 ns;

   constant S0price : std_logic_vector(7 downto 0) := "00000011"; --pret bautura 1 = 3 lei
   constant S1price : std_logic_vector(7 downto 0) := "00000101"; --pret bautura 2 = 5 lei

   constant un_leu : std_logic_vector(7 downto 0) := "00000001"; --bancnote de 1 leu
   constant cinci_lei : std_logic_vector(7 downto 0) := "00000101"; --bancnote de 5 lei
   constant zece_lei : std_logic_vector(7 downto 0) := "00001010"; --bancnote de 10 lei

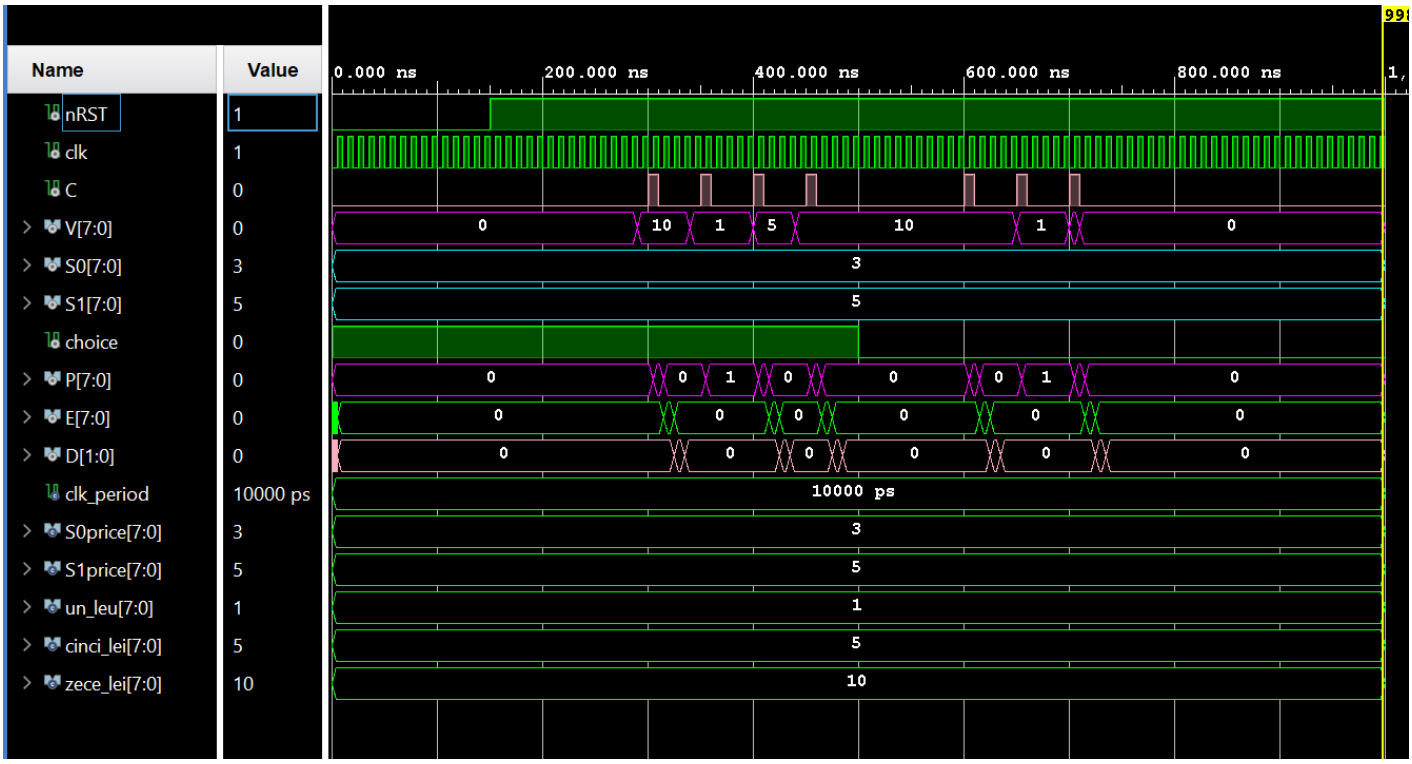

BEGIN

  -- Instantiere UUT(Unit Under Test)
   uut: vending_machine PORT MAP (
          nRST => nRST,
          clk => clk,
          C => C,
          V => V,
          S0 => S0,
          S1 => S1,
          choice => choice,
          P => P,
          E => E,
          D => D
        );

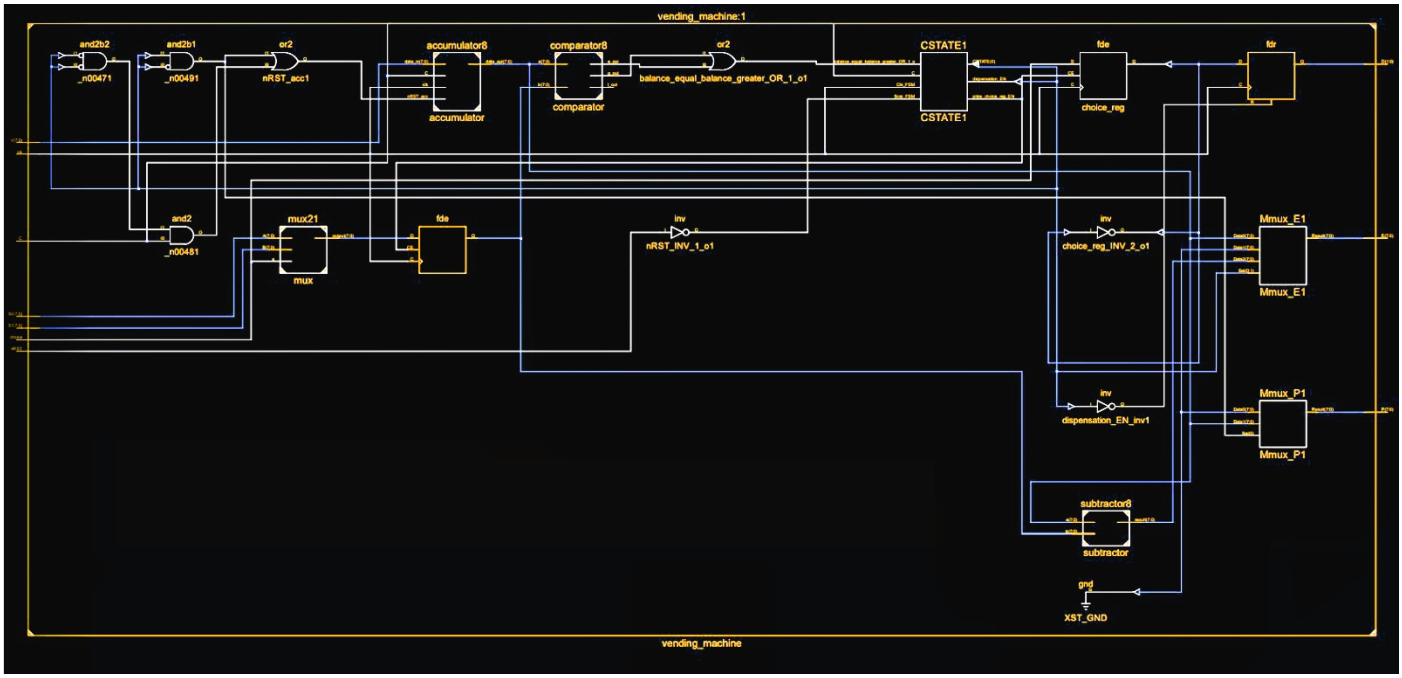
   CLK <= not(CLK) after clk_period/2;
   nRST <= '0', '1' after 150 ns;  -- mentine starea de resetare pentru 400 ns.
   s0 <= S0price; --pret bautura 1 = 3 lei
   S1 <= S1price; --pret bautura 2 = 5 lei
   choice <= '1', '0' after 500 ns;
   C <= '0', '1' after 300 ns, '0' after 310 ns, '1'after 350ns, '0' after 360 ns,
             '1' after 400 ns, '0' after 410 ns, '1'after 450ns, '0' after 460 ns,
             '1' after 600 ns, '0' after 610 ns, '1'after 650ns, '0' after 660 ns,
             '1' after 700 ns, '0' after 710 ns;

   v <= (others => '0'), zece_lei after 290 ns, un_leu after 340 ns, cinci_lei after 400
ns, zece_lei after 440 ns,
   zece_lei after 600 ns, un_leu after 650 ns, cinci_lei after 700 ns, (others => '0')
after 710 ns;
END;
```

SIMULARE



DIAGRAMĂ STĂRI



ACCUMULATOR8

```
library ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity accumulator8 is
    port(
        clk : in std_logic;
        nRST_acc : in std_logic;
        C : in std_logic;   --devine 1 cand detecteaza o bancnota
        data_in : in std_logic_vector(7 downto 0);
        data_out : out std_logic_vector(7 downto 0)
        );
end accumulator8;

architecture rtl of accumulator8 is
component adder8 IS
  port(
    a : in std_logic_vector(7 downto 0);
    b : in std_logic_vector(7 downto 0);
    c_in: in std_logic;
    s: out std_logic_vector(7 downto 0);
    c_out: out std_logic
    );
end component;

signal temp1 : std_logic_vector(7 downto 0);
signal temp2 : std_logic_vector(7 downto 0);
signal c_in : std_logic;
signal c_out : std_logic;

begin
    c_in <= '0';

    adder : adder8 port map (data_in, temp2, c_in, temp1, c_out);

    reg: process(clk)
    begin
        if (clk'event and clk = '1') then
            if nRST_acc = '0' then
                temp2 <= (others => '0');
            elsif (C = '1') then
                temp2 <= temp1;
            end if;
        end if;
    end process reg;

    data_out <= temp2;
end rtl;
```

ADDER8

```
library ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

ENTITY adder8 IS
  PORT(
    a : in std_logic_vector(7 downto 0);
    b : in std_logic_vector(7 downto 0);
    c_in: in std_logic;
    s: out std_logic_vector(7 downto 0);
    c_out: out std_logic);
end adder8;

architecture model of adder8 is
signal c: std_logic_vector(7 downto 1) := (others => '0');
    COMPONENT full_adder
      PORT(
         a : IN STD_LOGIC;
         b : IN STD_LOGIC;
         c_in : IN STD_LOGIC;
         sum : OUT STD_LOGIC;
         c_out : OUT STD_LOGIC);
    END COMPONENT;
BEGIN
    bit0 : full_adder PORT MAP (a(0),b(0),c_in,s(0),c(1));
    bit1 : full_adder PORT MAP (a(1),b(1),c(1),s(1),c(2));
    bit2 : full_adder PORT MAP (a(2),b(2),c(2),s(2),c(3));
    bit3 : full_adder PORT MAP (a(3),b(3),c(3),s(3),c(4));
    bit4 : full_adder PORT MAP (a(4),b(4),c(4),s(4),c(5));
    bit5 : full_adder PORT MAP (a(5),b(5),c(5),s(5),c(6));
    bit6 : full_adder PORT MAP (a(6),b(6),c(6),s(6),c(7));
    bit7 : full_adder PORT MAP (a(7),b(7),c(7),s(7),c_out);
END model;
```

COMPARATOR

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity comparator_1bit is
    port(
        a : in std_logic;
        b : in std_logic;
        g : out std_logic;
        e : out std_logic;
        l : out std_logic);
end comparator_1bit;

architecture structural of comparator_1bit is
```

```
begin
    g <= (a and (not b));
    e <= (a xnor b);
    l <= ((not a) and b);
end structural;
```

COMPARATOR_1BIT

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity comparator_1bit is
    port(
        a : in std_logic;
        b : in std_logic;
        g : out std_logic;
        e : out std_logic;
        l : out std_logic);
end comparator_1bit;

architecture structural of comparator_1bit is
begin
    g <= (a and (not b));
    e <= (a xnor b);
    l <= ((not a) and b);
end structural;
```

COMPARATOR8

```
library ieee;
use ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity comparator8 is
    port(
        a : in std_logic_vector(7 downto 0);
        b : in std_logic_vector(7 downto 0);
        g_out : out std_logic;
        e_out : out std_logic;
        l_out : out std_logic);
end comparator8;

architecture structural of comparator8 is
    signal g : std_logic_vector(7 downto 0);
    signal e : std_logic_vector(7 downto 0);
    signal l : std_logic_vector(7 downto 0);

    component comparator_1bit
    port(
        a : in std_logic;
        b : in std_logic;
```

```
        g : out std_logic;
        e : out std_logic;
        l : out std_logic);
    end component;

    component comparator_CO  --comparator_consider_order
    port(
        g_old : in std_logic;
        e_old : in std_logic;
        l_old : in std_logic;
        a : in std_logic;
        b : in std_logic;
        g : out std_logic;
        e : out std_logic;
        l : out std_logic);
    end component;

begin
    bit7 : comparator_1bit port map (a(7), b(7), g(6), e(6), l(6));
    bit6 : comparator_CO port map (g(6), e(6), l(6), a(6), b(6), g(5), e(5), l(5));
    bit5 : comparator_CO port map (g(5), e(5), l(5), a(5), b(5), g(4), e(4), l(4));
    bit4 : comparator_CO port map (g(4), e(4), l(4), a(4), b(4), g(3), e(3), l(3));
    bit3 : comparator_CO port map (g(3), e(3), l(3), a(3), b(3), g(2), e(2), l(2));
    bit2 : comparator_CO port map (g(2), e(2), l(2), a(2), b(2), g(1), e(1), l(1));
    bit1 : comparator_CO port map (g(1), e(1), l(1), a(1), b(1), g(0), e(0), l(0));
    bit0 : comparator_CO port map (g(0), e(0), l(0), a(0), b(0), g_out, e_out, l_out);

end structural;
```

FULL_ADDER

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

ENTITY full_adder IS
  PORT(
    a : IN STD_LOGIC;
    b : IN STD_LOGIC;
    c_in : IN STD_LOGIC;
    sum : OUT STD_LOGIC;
    c_out : OUT STD_LOGIC);
END full_adder;

ARCHITECTURE model OF full_adder IS
BEGIN
 sum <= a XOR b XOR c_in;
 c_out <= (a AND b) OR (c_in AND (a XOR b));
END model;
```

MUX21

```
library ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity mux21 is
    port(
        A : in std_logic_vector(7 downto 0);
        B : in std_logic_vector(7 downto 0);
        s : in std_logic;
        output : out std_logic_vector(7 downto 0)
        );
end mux21;

architecture model of mux21 is
begin
    output <= A when (s = '0') else
                B when (s = '1');
end model;
```

SUBTRACTOR8

```
library ieee;
USE ieee.std_logic_1164.all;
use IEEE.NUMERIC_STD.ALL;

entity subtractor8 is
    port(
        a : in std_logic_vector(7 downto 0);
        b : in std_logic_vector(7 downto 0);
        result : out std_logic_vector(7 downto 0)
        );
end subtractor8;

architecture structural of subtractor8 is
component adder8 IS
  port(
    a : in std_logic_vector(7 downto 0);
    b : in std_logic_vector(7 downto 0);
    c_in: in std_logic;
    s: out std_logic_vector(7 downto 0);
    c_out: out std_logic
    );
end component;

signal c_in : std_logic;
signal c_out : std_logic;
signal not_b : std_logic_vector(7 downto 0);
```

```
begin
    not_b <= (not b);
    c_in <= '1';
    subtractor : adder8 port map (a, not_b, c_in, result, c_out);
end structural;
```